

Project Guidelines - AI Dairy

Efraim Safrai

June 2026

Project Guidelines

This is a really nice idea, and is a good solution I wished existed during my undergrad. I can't wait to see this implemented, because there is not much backend complexity involved in this project compared to other projects. I will suggest a few more features to make this project well rounded and complete with all we will learn this semester. Hopefully these features make sense with the vision you had for the project, if they don't, feel free to change to another similar feature that will test the same concepts. But **Let me know via mail/whatsapp if you want to change these guidelines, it is ok to change them but I need to know about it.**

THIS PROJECT IS 80% OF YOUR GRADE

IMPORTANT NOTE: these are the guidelines needed for a project to be at a score of 100, you do not have to complete them all and we do not expect you to complete them all. Remember that each bug we find is -5 points loss and each week of delay in the submission is -5 points loss. It is completely reasonable to only complete some of the features in the guidelines. You will get partial credit for a partial completion of each feature.

The guidelines you have set yourself are as follows:

- **60 points - Project Proposal:** Implement a working, localhost web application that implements your proposed idea, it should include all that was mentioned in the proposal, **and you will be checked accordingly.**
- - Users, password and authentication system, make sure is secure.
 - A chat with AI, that can take requests in free speech "e.g. remind me to water my plant", and the AI (import Gemini API), will add to the calander what needs to be added (e.g. figure out a schedule for this).
 - I can ask the chat what I need to do today/this week and it will sum it up for me.
 - The user can himself change the calander as he wants by hand from the app.
 - The clients will get notifications according to the schedule of their calander.
- **Long-Term memory, 5 points:** Make sure you implement a long-term memory system for each user, each user should have a file about him in the database, his wants and how best to talk to him, What goes on my day and my preferences (do easy task or hard tasks first). You can even ask the student some questions during sign up to help the AI know what courses interest him. The server should save **all necessary information** to be as personal as possible for the student. In short, it should save everything that is needed to make sure the AI won't "forget" something critical about the student's preferences. To do this correctly, you will have to implement a database where the AI can choose when information should be concluded and saved to the database, and when it can be discarded. You will use this system to stop easy hallucinations that can happen from loss of information about the student. **Possible example:** If the user tells the AI that it feels unwell and can't do much this week, the AI will remember to take that into account when giving the user what to do today. **The AI's long memory will be checked by us.**

- **Robustness to Hallucinations, 5 points:** The AI should be robust to hallucinations, and should not ever recommend some course that doesn't exist, or say that some course contains material that the student is interested in when it doesn't. The way to do this is to keep a database of all courses, their content, their reviews and their explanations, and make sure the AI always checks the database before answering any question about a course. The AI should also be robust to hallucinations in the student's text, and should not take the student's feedback as a fact without checking it against the database.
- **Shared Calenders, 10 points:** Implement a feature that allows users to make a shared calender with other users, and they can both edit the calender and see each other's tasks. Each user can use his chat to alter the shared calender. And there will be long term memory for that calender (personality file for each calender), the chat will also be shared.
- **Local LLM use instead of API calls, 10 points:** Because your project will require many API calls, your options for an external API are limited - the free options are too weak to not hallucinate easily and the strong options have a very small amount of token use available. Therefore, relying on a free external API will not be enough to make your project work. You should download a local LLM that will be able to handle the complex conversations and the huge amount of data to read and remember. And you should use the local LLM to handle everything. You can use any open source LLM you want, but make sure it is strong enough to have complex mathematical conversations when needed and to not easily hallucinate what it read from the database. Make sure you use parallel programming techniques to handle the local LLM and the priority queue for the chat, the product should work well even when many students are chatting at once.
- **Online Forum, 10 points:** Implement an Online forum, it should be kind of similar to a social network/blog. People on this page could through their client share Ideas, their achievements, some tips for the degree, ideas to make the app better etc. you will have the following features for the online forum (feel free to add/change slightly):
 1. Each client should be able to post on the forum, and each post should be visible to all the other clients. Each post should have a title, a body, and the option to add images and videos to the post as well.
 2. Each client should have an option to post anonymously, and if they choose to do so, their name should not be visible to the other clients.
 3. Each client should be able to comment on each post, and each comment should be visible to all the other clients. Each comment should have a body, and the option to add images and videos to the comment as well.
 4. Each client should be able to like/dislike each post and each comment, and see how many likes each post and comment has, he should also be able to look at his posts on his personal area, and see how many likes and dislikes he has overall.

5. There should be a direct-messaging feature between the clients, where each client can send a message to any other client, and the message should be visible only to the sender and the receiver. Each message should have a body, and the option to add images and videos to the message as well.
6. The clients will get a notification on the webapp when they receive a new direct message, they will also be able to see who liked/disliked their posts and comments.
7. A person can send and share a new "event" that you can add to your own calander.
8. **IMPORTANT:** You should think about how to protect your system againsts spamming (for example, a user sending 1000 messages in a short time to overload the server), and against sending huge video files to overload the database.
9. **Cold seeding:** to get the ball rolling, the website should be created with already a few "fake" clients, with a few posts and comments on the forum.

This can be implemented in any way you want, and you can add and change the proposed features as you wish (let me know if you are doing a major change) but it should be possible to send messages on this forum and receive them in live feedback (no page refresh should be necessary), the chats should be saved and retrievable for the users, and each user should get to access their chat history and get a live notification in the webapp for a new message. The users have a lot of freedom here, so make sure your server cannot be easily crashed by malicious users, and that you have a good security system in place for this online forum.

10 points for implementing the feature, partial scoring will be given for partial implementation.